

第五章 运行时环境

冯洋
南京大学计算机学院
fengyang@nju.edu.cn

运行时刻环境

- 动机
 - 编译器需准确实现源程序语言中包含的各个抽象概念
 - 名字，作用域，绑定，数据类型，运算符，过程，参数，控制流构造等
 - 编译器还需和操作系统及其他系统软件协作，在目标机上支持这些抽象概念
 - 为做到这一点，编译器创建并管理一个运行时刻环境，它编译得到的目标程序就运行在这个环境中
 - 运行时环境需要处理诸多事务
 - 如为源程序中命名的对象分配和安排存储位置
 - 确定目标程序访问变量时使用的机制
 - 过程间的连接，参数传递机制，及与操作系统，输入输出设备及其他程序的接口

存储相关概念

- 逻辑地址管理
 - 对逻辑地址空间的管理和组织是由编译器，操作系统，目标机共同完成
 - 操作系统将逻辑地址映射为物理地址，物理地址对整个内存空间编址
- 内存组织
 - 假定运行时刻存储是以多个连续字节块的方式出现的，其中字节是内存的最小编址单元
 - 一个字节包含 8 个二进制位，4 个字节构成一个机器字
 - 多字节数据对象总是存储在一段连续的字节中，并把第一个字节作为它的地址

提纲

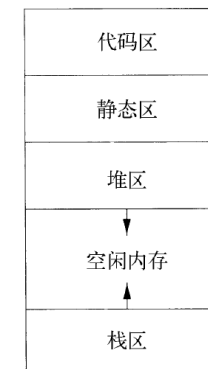
- 运行时刻环境
- 存储管理
 - 栈分配
 - 堆管理
 - 垃圾回收
- Java GC设计
- 运行时环境图解

运行时刻环境

- 运行时刻环境
 - 为数据分配安排存储位置
 - 确定访问变量时使用的机制
 - 过程之间的连接
 - 参数传递
 - 和操作系统、输入输出设备相关的其它接口
- 主题
 - 存储管理：栈分配、堆管理、垃圾回收
 - 对变量、数据的访问

存储分配的典型方式

- 数据对象的存储布局受目标机的寻址约束的影响很大。
 - 因为对齐的原因而产生的闲置空间称为补白
- 目标程序的代码放置在代码区
 - 大小在编译时刻就已经固定下来了
 - 编译器可将可执行目标代码放在一个静态确定的区域，即代码区
 - 这个区常位于存储的低端

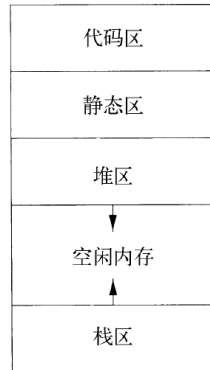


存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值

- 静态区

- ❑程序的某些数据对象的大小可以在编译时刻知道，可被放置在另一个称为静态区的区域
- ❑该区域可被静态确定
- ❑放置的数据对象包括全局常量和编译器产生的数据，如用于支持垃圾回收的信息等

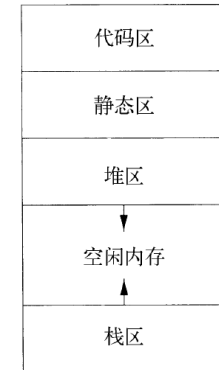


存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值

- 栈和堆

- ❑为了将运行时刻的空间利用率最大化，栈和堆被放在剩余地址空间的相对两端
- ❑这两个区域是动态的，根据需要会向对方增长
- ❑栈区用来存放称为活动记录的数据结构，这些活动记录在函数调用过程中生成
- ❑实践中，栈向较低地址方向增长，堆向较高地址方向增长
- ❑我们假定栈是由低地址向高地址增长，以便我们能在示例中方便地使用正的偏移量



静态和动态存储分配

- 静态和动态分别表示编译时刻和运行时刻
- 静态分配
 - 编译器在编译时刻就可以做出存储分配决定，不需要考虑程序运行时刻的情形
 - 全局变量
- 动态分配
 - 栈式存储
 - ❑和过程的调用/返回同步进行分配和回收，值的生命期和过程生命期相同
 - ❑一个过程的局部名字在栈中分配空间
 - 堆存储：数据对象比创建它的过程调用存活时间更长
 - ❑手工进行回收：C++/C...
 - ❑垃圾回收机制：Java/C#/Python...

栈式分配

- 主要内容
 - 活动树
 - 活动记录
 - 调用代码序列
 - 栈中的变长数据

活动树

- 有些语言使用过程，函数或方法作为用户自定义动作的单元
 - 几乎所有针对这些语言的编译器都把它们运行时刻存储当做一个栈来管理
- 过程调用（过程活动）在**时间**上总是嵌套的
 - 后调用的先返回
 - 因此用栈式分配来分配过程活动所需内存空间
- 程序运行的所有过程活动可以用树表示
 - 每个结点对应于一个过程活动
 - 根结点对应于main过程的活动
 - 过程p的某次活动对应的结点的所有子结点
 - ❑此次活动所调用的各个过程活动（从左向右，表示调用的先后顺序）

活动树的例子（1）

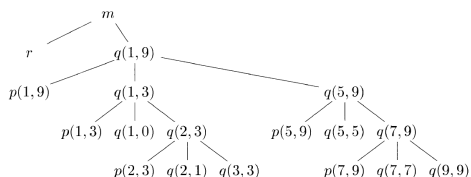
- 程序：如右图所示，p代表partition，q代表quicksort
 - 如果过程p的一个活动调用了过程q，则q的该次活动必定在p的活动结束之前结束。有三种常见的情况
 - ❑q的该次活动正常结束，则基本上任何语言中，控制流从p中调用q的点之后继续
 - ❑q的该次活动直接或间接地中止了，即不能再继续执行了，此时，q和p同时结束
 - ❑q的该次活动因为q不能处理的某个异常而结束。过程p可能会处理这个异常，此时q的活动已经结束而p的活动继续执行。如果p不能处理这个异常，那么p的活动和q的活动一起结束

```
enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
  leave quicksort(1,3)
  enter quicksort(5,9)
  ...
  leave quicksort(5,9)
  leave quicksort(1,9)
leave main()
```

活动树的例子 (1)

- 程序: p代表partition, q代表quicksort, r代表readArray

➤可用一棵树来表示在整个运行期间的所有过程的活动, 这棵树称为活动树, 树中的每个结点对应于一个活动, 根结点是启动程序执行的main过程的活动



```
enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
    leave quicksort(1,3)
    enter quicksort(5,9)
    ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()
```

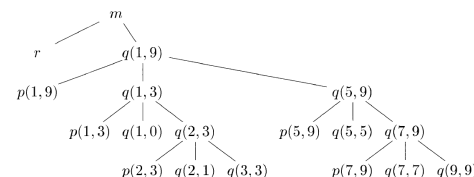
活动树的例子 (1)

- 程序: p代表partition, q代表quicksort, r代表readArray

➤活动树

□过程p的某个活动的结点上, 其子结点对应于被p的调用的各个过程的活动

□按活动被调用的顺序, 自左向右地显示。一个子结点需在其右兄弟结点的活动开始之前结束



```
enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
    leave quicksort(1,3)
    enter quicksort(5,9)
    ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()
```

活动树的例子 (1)

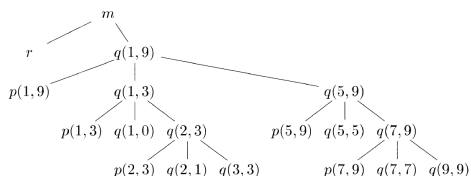
- 程序

➤活动树与程序行为对应关系

□过程调用的序列和活动树的前序遍历相对应。

□过程返回的序列和活动树的后序遍历相对应。

□假定控制流位于某个过程的特定活动中, 且该过程活动对应于活动树上的某个结点N。则当前尚未结束的就是结点N及其祖先结点的活动



```
enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
    leave quicksort(1,3)
    enter quicksort(5,9)
    ...
    leave quicksort(5,9)
  leave quicksort(1,9)
leave main()
```

活动记录

- 过程调用和返回由控制栈进行管理

➤每个活跃的活动对应于栈中的一个活动记录

- 活动记录按照活动的开始时间, 从栈底到栈顶排列

➤活动树的根位于栈底

➤栈中全部活动记录的序列对应于在活动树中到达当前控制所在的活动结点的路径

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量

活动记录

- 控制栈的结构

➤假定栈底在栈顶上

➤活动记录可能出现的各种类型数据

□临时值: 如表达式求值过程中产生的中间结果无法存放在寄存器中时, 就会生成这些临时值

□局部数据: 对应于这个活动记录的过程的局部数据

□机器状态: 包括此次调用之前的机器状态信息, 如返回地址和寄存器的内容

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量

活动记录

- 控制栈的结构

➤活动记录可能出现的各种类型数据

□访问链: 被调用过程需其他地方的某个数据时需要使用访问链进行定位

□控制链: 指向调用者的活动记录

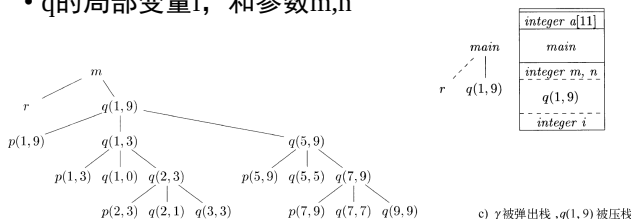
□返回值: 当被调用函数有返回值时, 要有一个用于存放这个返回值的空间

□实在参数: 调用过程使用的实在参数。这些值通常将尽可能放在寄存器中, 而非活动记录中

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量

运行时刻栈的例子

- a[11]为全局变量
- main没有局部变量
- r有局部变量i
- q的局部变量i, 和参数m,n



调用代码序列

- 调用代码序列(calling sequence)为活动记录分配空间
 - 实现过程调用的代码段
 - 这个代码序列为一个活动记录在栈中分配空间, 并在此记录的字段中填写信息
- 返回代码序列(return sequence)恢复机器状态, 使调用者继续运行
 - 一段代码序列, 它恢复机器状态, 使得调用过程能在调用结束之后继续执行
- 调用代码序列会分割到调用者和被调用者中
 - 根据源语言、目标机器、操作系统的限制, 可以有不同的分割方案
 - 在分割运行时刻任务时, 调用者和被调用者之间不存在明确界限
 - 如果一个过程在n个不同点上被调用, 分配给调用者的那部分调用代码序列会被生成n次
 - 把代码尽可能放在被调用者中

调用/返回代码序列的要求

- 数据方面
 - 能够把参数正确地传递给被调用者
 - 能够把返回值传递给调用者
- 控制方面
 - 能够正确转到被调用过程的代码开始位置
 - 能够正确转回调用者的调用位置 (的下一条指令)
- 调用代码序列和活动记录的布局相关

活动记录的布局原则

- 调用者和被调用者之间传递的值放在被调用者活动记录的开始位置
 - 靠近调用者的活动记录
 - 调用者能计算该次调用的实在参数的值, 并将它放在自身活动记录的顶部
 - 不用创建整个被调用者的活动记录
- 固定长度的项放在中间位置
 - 控制链、访问链、机器状态字段
 - 如果将机器状态信息标准化, 则错误发生时, 诸如调试器等程序可将栈中的内容解码

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量

活动记录的布局原则

- 编译时大小未知的项在活动记录尾部
 - 一部分局部变量具有固定的长度, 编译器通过检查该变量的类型就可确定其长度
- 栈顶指针(top_sp)通常指向固定长度字段的末端
 - 固定长度的数据就可通过固定的相对于栈顶指针的偏移量来访问
 - 而中间代码生成器知道并可使用这些偏移量
 - 后果
 - 活动记录中的变长域实际上位于栈顶之上
 - 它们的偏移量需运行时刻计算

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量

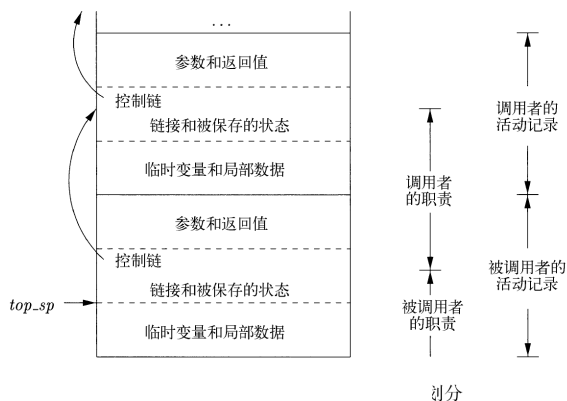
调用代码序列的例子

- 调用代码序列 (Calling sequence)
 - 调用者计算实在参数的值
 - 将返回地址和原top_sp存放放到被调用者的活动记录中。调用者增加top_sp的值 (越过了局部数据、临时变量、被调用者的参数、机器状态字段)
 - 被调用者保存寄存器值和其他状态字段
 - 被调用者初始化局部数据、开始运行
- 返回代码序列 (Return sequence)
 - 被调用者将返回值放到和参数相邻的位置
 - 恢复top_sp和寄存器, 跳转到返回地址

调用者/被调用者的活动记录

- 调用代码序列及它在调用者和被调用者之间的划分描述如下

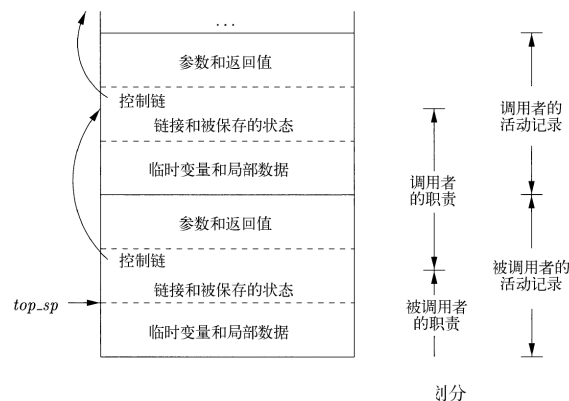
- 调用者计算实在参数的值
- 调用者将返回地址和原来的top_sp值存放到被调用者的活动记录中
- 被调用者保存寄存器值和其他状态信息
- 被调用者初始化其局部数据并开始执行



调用者/被调用者的活动记录

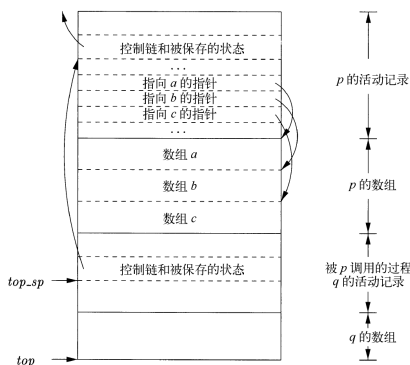
- 与此匹配的返回代码序列如下

- 被调用者将返回值放到与参数相邻的位置
- 使用机器状态字段中的信息，被调用者恢复top_sp和其他寄存器，然后跳转到由调用者放在机器状态字段中的返回地址
- 尽管top_sp已经被减少，但调用者知道返回值相对于当前top_sp值的位置



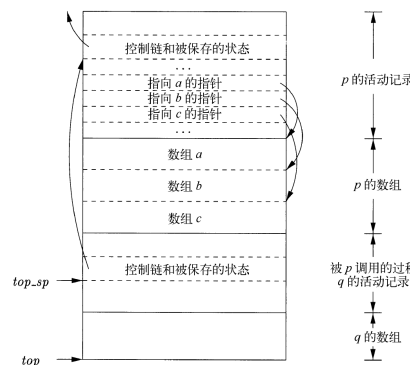
栈中的变长数据

- 运行时刻存储管理系统需频繁地处理某些数据对象的空间分配
 - 将未知大小的对象，数组及其他结构分配在栈中
- 如果数据对象的生命期局限于过程活动的生命期，就可以分配在运行时刻栈中
 - 变长数组也可以放在栈中
- top: 指向实际的栈顶
- top_sp: 用于寻找顶层记录的定长字段



栈中的变长数据

- 数据对象生命期局限于过程活动内，就可以分配在运行时刻栈中
 - 尽管这些数组的存储出现在栈中，它们并不是p的活动记录的一部分
 - p执行时，这些指针的位置相对于栈顶指针的偏移量是已知的
 - 栈中数据的访问通过指针top和top_sp完成
 - top标记了实际的栈顶位置，它指向下一个活动记录将开始的位置
 - 第二个指针top_sp用来找到顶层活动记录的局部的定长字段



运行时环境——图解

- 运行时环境示例

一个简单的例子

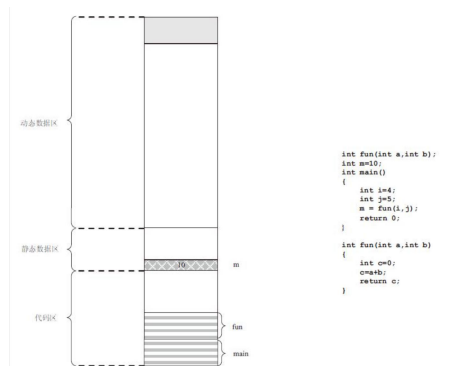
- 两个函数
- 一个全局变量m
- 若干局部变量i, j, c, a...
- 一次函数调用

```
int fun(int a, int b);
int m=10;
int main()
{
    int i = 4;
    int j = 5;
    m = fun(i,j);
    return 0;
}

int fun(int a, int b)
{
    int c = 0;
    c = a + b;
    return c;
}
```

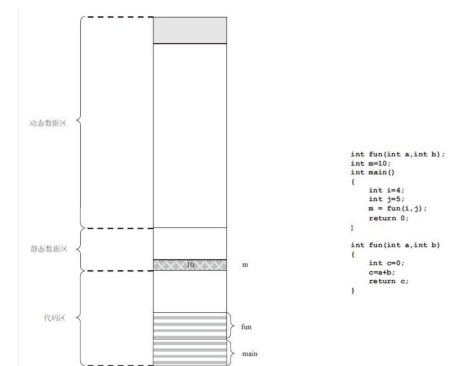
一个简单的例子

- 整个程序放到了内存中的样子如右图。共有三个区域：代码区，静态数据区，动态数据区



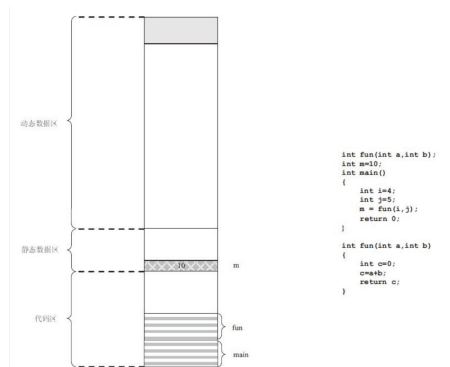
一个简单的例子

- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区



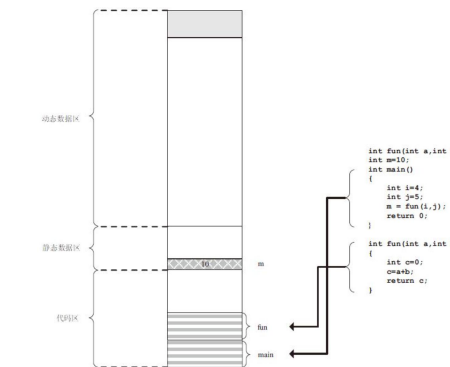
一个简单的例子

- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区



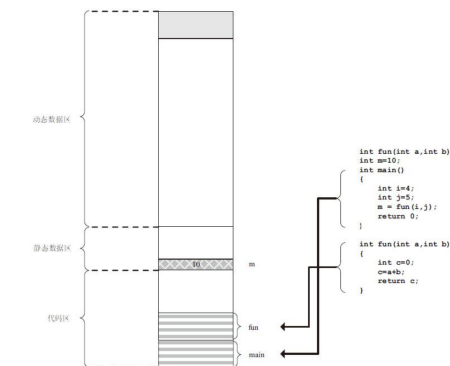
一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据



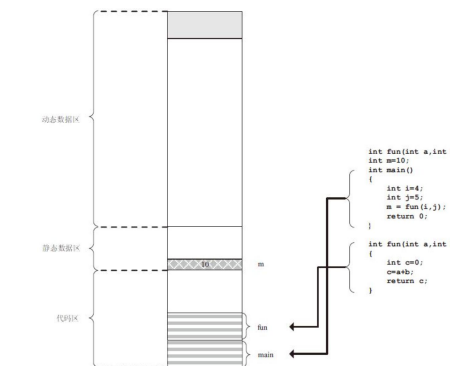
一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据；只有在程序开始执行后，在指令的驱动下，这一区域才会产生数据，压栈和清栈的工作就是在这一区域完成的



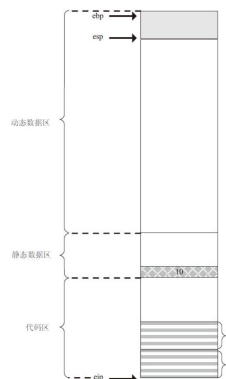
一个简单的例子

- 程序执行的本质，就是代码区的指令不断执行
 - 驱使动态数据区和静态数据区产生数据变化



一个简单的例子

- 我们假定CPU 中有三个寄存器，分别是 eip、ebp、esp。其中eip 永远指向代码区要执行的下一条指令
 - ebp与esp的位置在程序加载时由内核设置
 - eip的位置一开始指向main函数
- ebp和esp用来管理栈空间
 - ebp指向栈底，esp指向栈顶，代码区中的程序执行，函数调用、返回，均存在不断的压栈与清栈
 - 栈底画在栈顶的上方

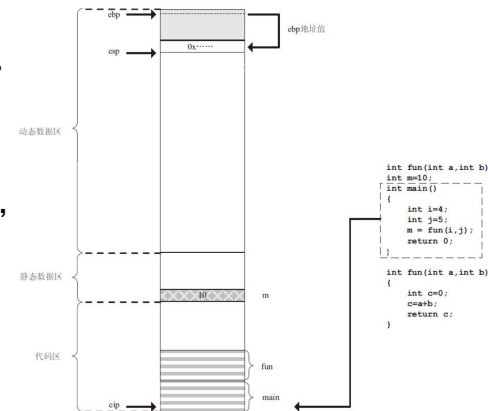


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

一个简单的例子

- 程序开始执行main函数第一条指令，eip自动指向下一条指令。第一条指令的执行，则使得ebp的地址值被保存到栈里面
- 保存的目的是本程序执行完毕后，ebp还能返回到现在的位置，复原现在的栈。ebp保存之后，esp则自动向栈顶移动，它永远指向栈顶

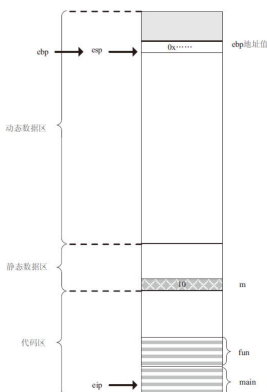


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

一个简单的例子

- 在对返回地址保存之后，开始为main构建栈，main的栈构建情况如右图所示
- 注意，此事ebp原来指向的值已经被存起来了，所以esp就自动向栈顶移动，同时，ebp需要用来指向main的栈底
- 此时由于没有数据在栈中，ebp和esp的位置其实是重叠的

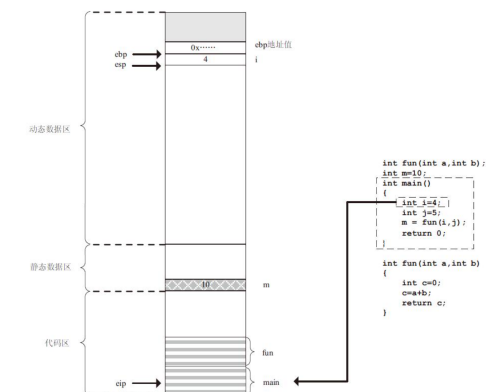


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

一个简单的例子

- 程序继续执行，第一个语句，int i = 4 被执行
- 注意到，对main来说，i 是一个局部变量，那么，其初始值4，被压入栈中
- 特别注意栈顶指针esp的变化

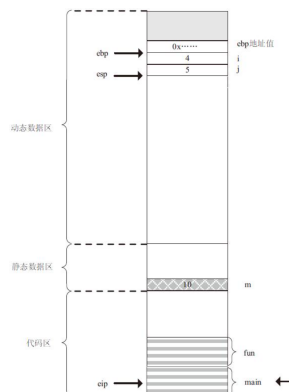


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

一个简单的例子

- 同样的道理，int j = 5被执行，5被压入到栈

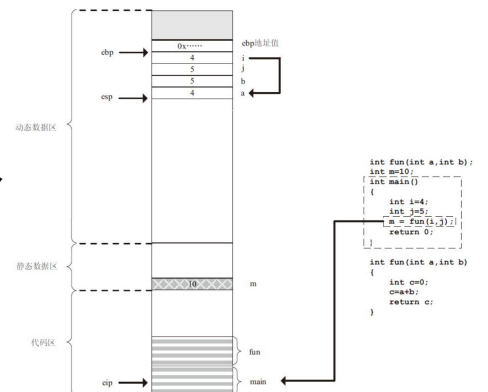


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

一个简单的例子

- 下一步，程序执行到调用函数fun的指令。特别注意，此处，我们对fun的调用，首先需要完成的，是传参指令
- 特别注意，在C语言中，参数入栈顺序与传参书写顺序正好相反。参数b先入栈，b的数值是局部变量j的数值5。因此，情况如右所示

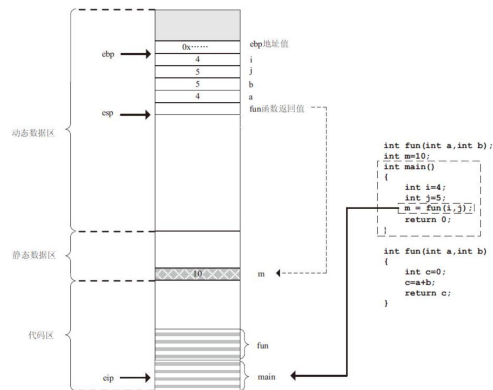


```
int fun(int a,int b);
int m=10;
int main()
{
    int i=4;
    int j=5;
    m = fun(i,j);
    return 0;
}

int fun(int a,int b)
{
    int c=0;
    c=a+b;
    return c;
}
```

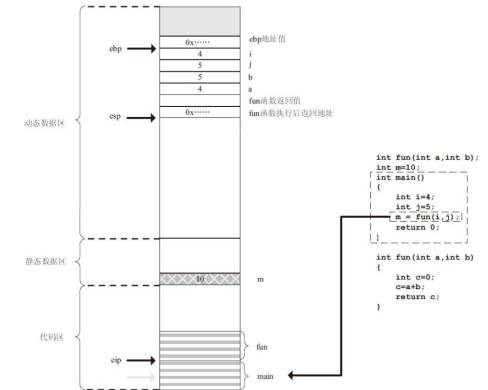
一个简单的例子

- 程序继续执行，那么应该压入的是fun函数的返回值，后面返回值应该是传递给m



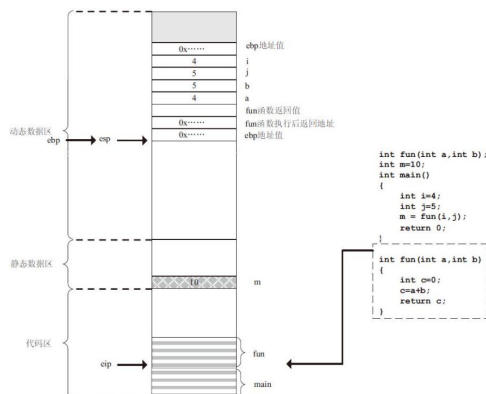
一个简单的例子

- 在保存好fun函数执行完毕的返回地址后，eip跳转到fun的代码区进行执行，如右图所示
- 后续则需要构建fun的栈



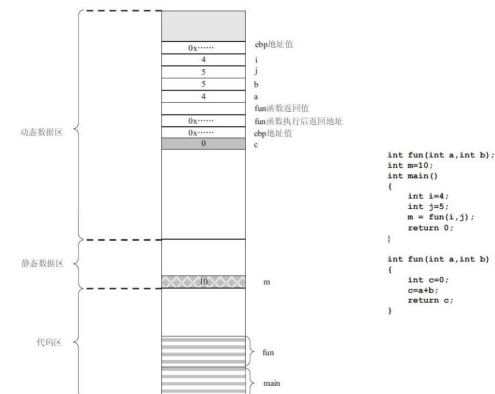
一个简单的例子

- 与前面我们调用main函数相似，ebp与esp再次指向同一个位置，此时fun的栈是空的



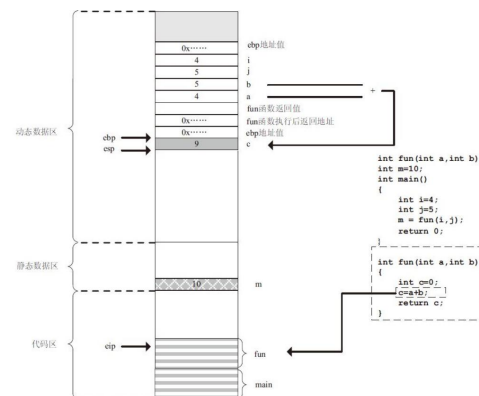
一个简单的例子

- 与前面的情况类似，c会被压入fun的栈中
- 后续是若干运算指令



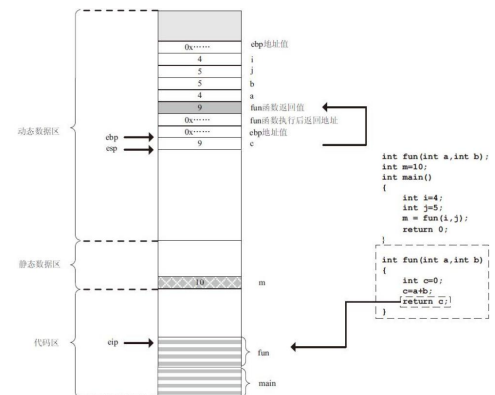
一个简单的例子

- 在执行a+b的过程中，我们可以看到，在运算指令的执行过程中，可以通过一系列的偏移计算，得到运算指令需要的数值
- 运算指令执行完毕后，赋值给局部变量c



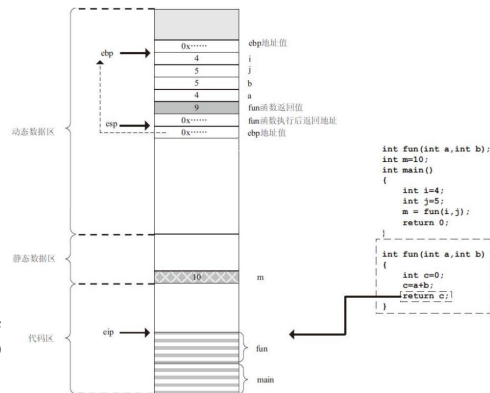
一个简单的例子

- 局部变量c的值作为返回值，对fun函数的返回值进行赋值



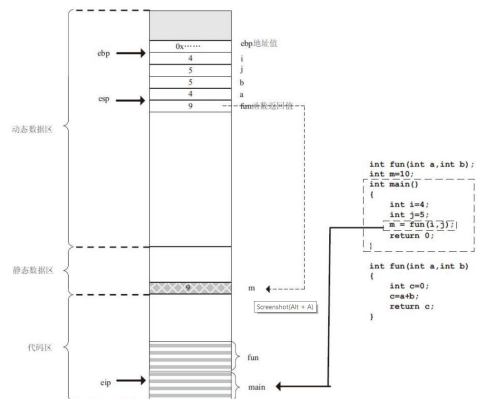
一个简单的例子

- fun 函数执行完毕，要恢复main 函数调用fun函数的现场
- 这一现场包括两个部分
 - 一部分是main函数的栈要恢复，包括栈顶和栈底
 - 另一部分是要找到fun函数执行后的返回地址，然后再跳转回到那里去继续执行
 - ebp的恢复相对简单，之前在此处有存储ebp的地址值，在消栈操作中，只需要取出该值赋值给ebp指针即可



一个简单的例子

- eip找到正确的后续执行指令之后，即将返回值赋值给正确的局部变量，即m。如右图所示

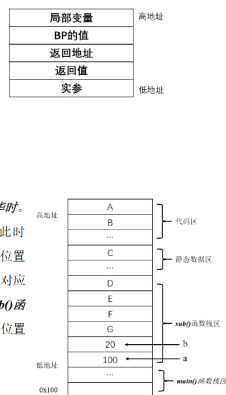


来，做个习题：

- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。
- (1) A: _____
- (2) B: _____
- (3) C: _____
- (4) D: _____
- (5) E: _____
- (6) F: _____
- (7) G: _____
- (8) SP: _____
- (9) BP: _____

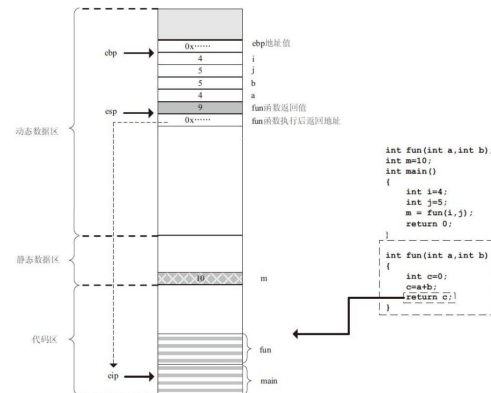
```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4)   int i = 100;
(5)   int j = 20;
(6)   m = sub(i, j);
(7)   printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11)   int res = 0;
(12)   res = a - b;
(13)   return res;
(14) }
```

假设程序运行完第 12 行，即 **res 计算完毕时**。请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 **sub()** 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 res 被定义时已经确定）。



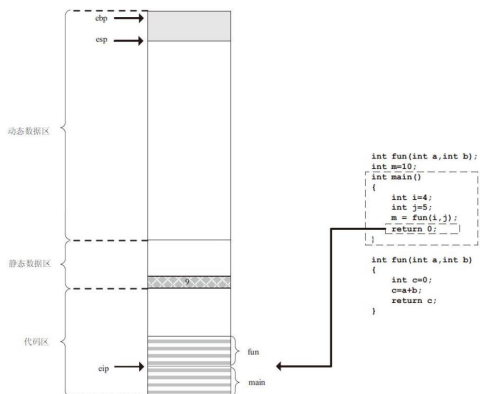
一个简单的例子

- 在ebp恢复后，esp自动退栈。
- 退栈后指向函数fun执行后的返回地址，然后执行ret指令
 - 该指令将esp此时的值反馈给eip，eip则可以找到main中的下一条指令，继续执行



一个简单的例子

- 同理，在执行完main相关的ret 指令之后，也需要对main函数的栈进行清除，清楚后的结果如右图所示
- 至此，整个程序执行完毕

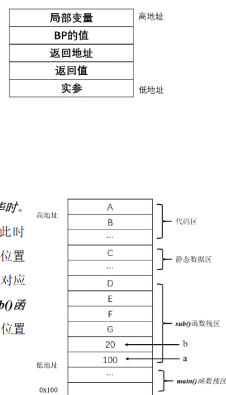


来，做个习题：

- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。
- (1) A: 代码 int i = 100
- (2) B: 代码 int j = 20
- (3) C: m = 0
- (4) D: 局部变量 res = 80
- (5) E: BP
- (6) F: 返回地址
- (7) G: 返回值 = null
- (8) SP: D 的上方
- (9) BP: a 下方

```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4)   int i = 100;
(5)   int j = 20;
(6)   m = sub(i, j);
(7)   printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11)   int res = 0;
(12)   res = a - b;
(13)   return res;
(14) }
```

假设程序运行完第 12 行，即 **res 计算完毕时**。请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 **sub()** 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 res 被定义时已经确定）。



非局部数据的访问（无嵌套过程）

- 没有嵌套过程时的数据访问
 - C语言中，每个函数能够访问的变量
 - 函数的局部变量：相对地址已知，且存放在当前活动记录内，top_sp指针加上相对地址即可访问
 - 全局变量：在静态区，地址在编译时刻可知
 - 很容易将C语言的函数作为参数进行传递
 - 参数中只需包括函数代码的开始地址
 - 在函数中访问非局部变量的模式很简单，不需要考虑过程是如何激活的

非局部数据的访问（嵌套声明过程）

- 语言允许嵌套地声明过程且仍然遵循通常的静态作用域规则时，数据访问变得比较复杂
 - 为一个内嵌过程p中的一个非局部名字x找出对应的声明是一个静态的决定过程
 - 将块结构的静态作用域规则进行扩展就可解决这个问题
 - 设x在一个外围过程q中声明
 - 根据p的一个活动找到相关的q的活动是一个动态的决定过程，需要额外的有关活动运行时刻信息

```
void A()
{
    int    x,y;
    void  B()
    {
        int b;
        x = b+y;
    }
    void  C() {B();}
    C();
    B();
}
```

当A调用C，C又调用B时： 当A直接调用B时：

A的活动记录	A的活动记录
C的活动记录	B的活动记录
B的活动记录	

非局部数据的访问（嵌套声明过程）

- 语言允许嵌套地声明过程且仍然遵循通常的静态作用域规则时，数据访问变得比较复杂
 - PASCAL中，如果过程A的声明中包含了过程B的声明，那么B可以使用在A中声明的变量
 - 当B的代码运行时，如果它使用A中的变量。那么这个变量指向运行栈中最上层的同名变量
 - 但是，我们不能通过嵌套层次直接得到A的活动记录的相对位置，必须通过访问链访问

```
void A()
{
    int    x,y;
    void  B()
    {
        int b;
        x = b+y;
    }
    void  C() {B();}
    C();
    B();
}
```

当A调用C，C又调用B时： 当A直接调用B时：

A的活动记录	A的活动记录
C的活动记录	B的活动记录
B的活动记录	

一个支持嵌套声明的例子

- ML是一种函数式语言
 - 变量一旦被声明并初始化就不会在改变，只有少数几个例外
- 定义变量并设定它们不可更改的初始值的语句有如下形式
 - val <name> = (expression)
- 函数使用如下语法进行定义
 - fun<name> (<arguments>) = <body>
- 使用下列形式的let语句来定义函数体
 - let<list of definitions> in <statements> end
 - 其中，定义通常是val或fun语句。
 - 每个这样的定义的作用域包括从该定义之后直到in为止的所有定义，及直到end为止的所有语句。函数可嵌套地定义，如函数p的函数体可能包括一个let语句，而该语句又包含了另一个函数q的定义

嵌套深度

- 嵌套深度是正文概念，可以根据源程序静态地确定
 - 不内嵌于任何其他过程中的过程，嵌套深度为1
 - 嵌套在深度为i的过程中的过程，深度为i+1

深度为1
sort
深度为2
readArray,
exchange,
quicksort
深度为3
partition

```
1) fun sort(inputFile, outputFile) =
    let
2)     val a = array(11,0);
3)     fun readArray(inputFile) = ... ;
4)     ... a ... ;
5)     fun exchange(i,j) =
6)         ... a ... ;
7)     fun quicksort(m,n) =
        let
8)         val v = ... ;
9)         fun partition(y,z) =
10)            ... a ... v ... exchange ...
11)        in
            ... a ... v ... partition ... quicksort
        end
12)    in
        ... a ... readArray ... quicksort ...
    end;
```

访问链

- 访问链
 - 当被调用过程需要其他地方的某个数据时需要使用访问链进行定位
 - 如果过程p在声明时嵌套在过程q的声明中，那么p的活动记录中的访问链指向最上层的q的活动记录
 - 从栈顶活动记录开始，访问链形成了一个链路，嵌套深度沿着链路逐一递减

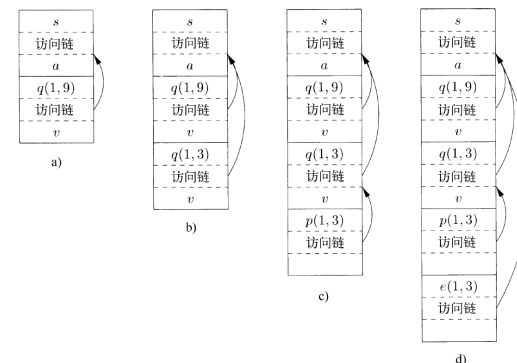
访问链

- 设深度为 n_p 的过程 p 访问变量 x ，而变量 x 在深度为 n_q 的过程中声明，则
 - $n_p - n_q$ 在编译时刻已知
 - 从当前活动记录出发，沿访问链前进 $n_p - n_q$ 次找到的活动记录中的 x 就是要找的变量位置
 - x 相对于这个活动记录的偏移量在编译时刻已知

访问链的例子

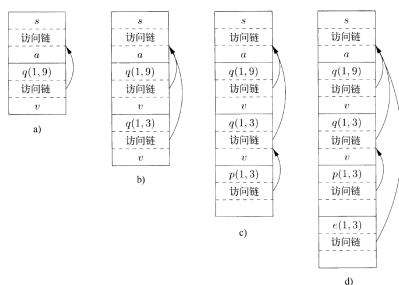
```

1) fun sort(inputFile, outputFile) =
    let
2)     val a = array(11,0);
3)     fun readArray(inputFile) = ... ;
4)     ... a ... ;
5)     fun exchange(i,j) =
6)         ... a ... ;
7)     fun quicksort(m,n) =
            let
8)         val v = ... ;
9)         fun partition(y,z) =
10)            ... a ... v ... exchange ...
11)        in
            ... a ... v ... partition ... quicksort
            end
12)    in
        ... a ... readArray ... quicksort ...
    end;
    
```



访问链的维护（直接调用过程）

- 当过程 q 调用过程 p 时，访问链的变化
 - p 的深度大于 q ：根据作用域规则， p 必然在 q 中直接定义；那么 p 的访问链指向当前活动记录
 - s 调用 $q(1,9)$
 - 递归调用 ($p=q$)：新活动记录的访问链等于当前记录的访问链
 - $q(1,9)$ 调用 $q(1,3)$
 - p 的深度小于等于 q 的深度：此时必然有过程 r ， p 直接在 r 中定义，而 q 嵌套在 r 中； p 的访问链指向栈最高的 r 的活动记录
 - p 调用 $exchange$



访问链的维护：访问链的定义+作用域规则

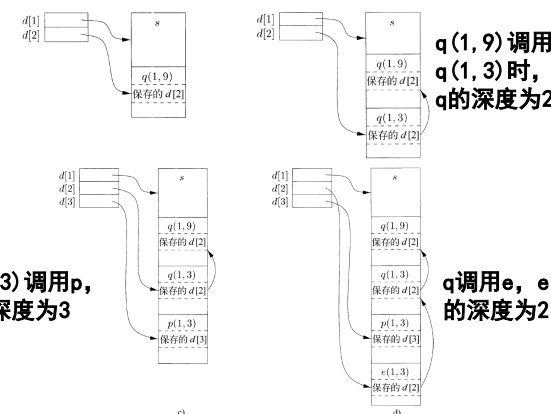
显示表

- 用访问链访问数据时，访问开销和嵌套深度差有关
- 使用显示表可以提高效率，访问开销为常量
- 显示表：数组 d 为每个嵌套深度保留一个指针
 - 指针 $d[i]$ 指向栈中最高的、嵌套深度为 i 的活动记录
 - 如果程序 p 中访问嵌套深度为 i 的过程 q 中声明的变量 x ，那么 $d[i]$ 直接指向相应的（必然是 q 的）活动记录
 - 注意： i 在编译时刻已知
- 显示表的维护
 - 调用过程 p 时，在 p 的活动记录中保存 $d[n_p]$ 的值并将 $d[n_p]$ 设置为当前活动记录
 - 从 p 返回时，恢复 $d[n_p]$ 的值

显示表的例子

```

1) fun sort(inputFile, outputFile) =
    let
2)     val a = array(11,0);
3)     fun readArray(inputFile) = ... ;
4)     ... a ... ;
5)     fun exchange(i,j) =
6)         ... a ... ;
7)     fun quicksort(m,n) =
            let
8)         val v = ... ;
9)         fun partition(y,z) =
10)            ... a ... v ... exchange ...
11)        in
            ... a ... v ... partition ... quicksort
            end
12)    in
        ... a ... readArray ... quicksort ...
    end;
    
```



堆管理

- 堆空间
 - 用于存放生命周期不确定、或生存到被明确删除为止的数据对象
 - 例如：new生成的对象可以生存到被delete为止
 - malloc申请的空间生存到被free为止
- 存储管理器
 - 分配/回收堆区空间的子系统
 - ❑ 分配：它尽可能使用堆中的空闲空间来满足分配请求。如没有请求大小的空间块可供分配，试图从操作系统中获得连续的虚拟内存来增加堆区的存储空间。如无可用空间，通知应用
 - ❑ 回收：通常不会将内存返回给操作系统，而是用于后续数据对象分配
 - 根据语言而定
 - ❑ C、C++需要手动回收空间
 - ❑ Java可以自动回收空间（垃圾收集）

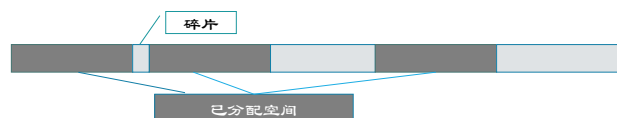
存储管理器

- 处理器通常具有少量寄存器，寄存器中的内容由软件控制
 - 有一层或多层高速缓存，通常用静态RAM制造，主存通常用动态RAM实现
 - 在一次内存访问中，首先在最近的存储中寻找数据，如找不到，到下一层寻找
- 数据以连续存储块的方式进行传输
 - 在主存和高速缓存之间的数据是按照被称为高速缓存线的块进行传输的
 - ❑ 高速缓存线的长度通常在32-256字节之间
 - ❑ 在虚拟内存和主内存之间的数据以“页”的内存块进行传输的大小通常在4-64KB之间

典型的大小		典型的访问时间
> 2GB	虚拟内存（磁盘）	3~15 ms
256MB~2GB	物理内存	100~150 ns
128KB~4MB	二级高速缓存	40~60 ns
16~64KB	一级高速缓存	5~10 ns
32个机器字	寄存器（处理器）	1 us

堆空间的碎片问题

- 随着程序分配/回收内存，堆区逐渐被割裂成为若干空闲存储块（窗口，hole）和已用存储块的交错
- 分配一块内存时，通常是把一个窗口的一部分分配出去，其余部分成为更小的块
- 回收时，被释放的存储块被放回缓冲池。通常要把连续的窗口接合成为更大的窗口



存储管理器

- 基本功能
 - 分配：为每个内存请求分配一段连续的、适当大小的堆空间
 - ❑ 从空闲的堆空间分配，如果失败，则从操作系统中获取内存、增加堆空间
 - 回收：把被回收的空间返回空闲空间缓冲池，以满足其他内存需求
- 评价存储管理器的特性
 - 空间效率：使程序需要的堆空间最小，即减小碎片
 - ❑ 通过使存储碎片达到最少而得到
 - 程序效率：充分运用内存系统的层次，提高效率
 - ❑ 关注对象在存储中的放置方法，更好地利用空间，并且有希望使程序运行得更快
 - 低开销：使分配/收回内存的操作尽可能高效

程序中的局部性

- 局部性
 - 时间局部性：一个程序访问的存储位置很可能将在一个很短的时间段内被再次访问
 - 空间局部性：访问过的存储位置的临近位置很可能在一个很短的时间段内被再次访问
 - 大部分程序表现出高度的局部性 – 程序的大部分运行时间花费在相对较小的一部分代码中，通常认为程序把90%的时间用来执行10%的代码
 - ❑ 程序经常含很多从来不执行的指令
 - ❑ 程序一次典型运行中，被调用的代码可能只有一小部分被实际执行
 - ❑ 通常的程序往往将大部分时间花费在执行程序中的最内层循环和最紧凑的递归环上
 - 利用存储层次结构的优化，将最近使用过的指令放入高速缓存的策略通常很有效
 - 还可改变数据布局或计算顺序，从而改变一个程序中的数据访问的时间局部性和空间局部性

堆空间分配方法

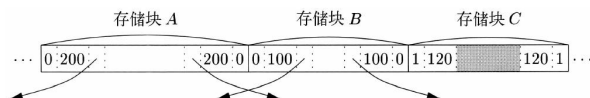
- Best-Fit
 - 总是将请求的内存分配在满足请求的最小的窗口中
 - 好处：可以将大的窗口保留下来，应对更大的请求
- First-Fit
 - 总是将对象放置在第一个能够容纳请求的窗口中
 - 放置对象时花费时间较少，但是总体性能较差
 - 但是first-fit的分配方法通常具有较好的数据局部性
 - ❑ 同一时间段内的生成的对象经常被分配在连续的空间内

使用容器的堆管理方法

- 设定不同大小的空闲块规格，相同规格的块放在同一容器中
- 较小的（较常用的）尺寸设置较多的容器
- 比如GNU的C编译器将所有存储块对齐到8字节边界
 - 空闲块的尺寸大小
 - 16, 24, 32, 40, ..., 512
 - 大于512的按照对数划分：每个容器的最小尺寸是前一个容器的最小尺寸的两倍
 - 荒野块：可以扩展的内存块
 - 分配方法
 - 对于小尺寸的请求，直接在相应容器中找
 - 大尺寸的请求，在适当的容器中寻找适当的空闲块
 - 可能需要分割内存块
 - 可能需要从荒野块中分割出更多的块

例子

- 相邻的存储块A、B、C
 - 当回收B时，通过对free/used位的查询，可知B左边的A是空闲的，而C不空闲
 - 同时还可以知道A、B合并为长度为300的块
 - 修改双重链表，把A替换为A、B接合后的空闲块
 - 注意：双重链表中一个结点的前驱并不一定是它邻近的块



正确的编程模式（1）

- 对象所有者（Object ownership）
 - 当一个对象的生命周期能被静态推导出来时，对象所有者的概念是很有用的
 - 每个对象总是有且只有一个所有者（指向此对象的指针）；只有通过Owner才能够删除这个对象
 - 当Owner消亡时，这个对象要么也被删除，要么已经被传递给另一个owner
 - 语句 `v=new ClassA`；创建的对象的所有者为v
 - 即将对v进行赋值的时刻（v的值即将消亡）
 - 要么v已经不是它所指对象的所有者；比如 `g=v` 可以把v的ownership传递给g
 - 要么需要在返回/赋值之前，执行 `delete v` 操作
 - 自动存储管理
 - 编程时需要了解各个指针在不同时刻是否owner
 - 防止内存泄漏，避免多次删除对象。不能解决悬空指针问题

管理和接合空闲空间

- 当回收一个块时，可以把这个块和相邻的块接合起来，构成更大的块
 - 为所有有固定尺寸的存储块保留一个容器，倾向于不把相邻的该尺寸的块合并成为双倍大小的块
 - 一个简单的分配/回收方案是维护一个位映射，其中的每个比特对应于容器中的一个块
 - 当一个块被回收时，将它对应的1改为0。如没有空闲块，就取一个新的页，将其分割为适当大小的存储块，同时扩展用于存储管理的位向量
- 支持相邻块接合的数据结构
 - 边界标记：在每一块存储块的两端，分别设置一个free/used位；相邻的位置上存放字节总数
 - 双重链接的、嵌入式的空闲块列表：列表的指针存放在空闲块中、用双向指针的方式记录了有哪些空闲块

处理手工存储管理

- 两大问题
 - 内存泄露：未能删除不可能再被引用的数据
 - 悬空指针引用：引用已被删除的数据
 - 把诸如读，写，回收等沿着一个指针试图使用该指针所指对象的所有操作称为对这个指针的“解引用”
- 其他问题
 - 空指针访问/数组越界访问
 - 访问非法地址，如对空指针解引用或者访问数组界限之外的元素
- 解决方法
 - 自动存储管理
 - 正确的编程模式

正确的编程模式（2）

- 引用计数
 - 每个动态分配的对象附上一个计数：记录有多少个指针指向这个对象
 - 在赋值/返回/参数传递时维护引用计数的一致性
 - 在计数变成0之时删除这个对象
 - 可以解决悬空指针问题；但是在递归数据结构中仍然可能引起内存泄漏
 - 不能发现无用的循环数据结构，当对象不能被访问，但因为它们相互引用，因此均不释放
 - 需要较大的运行时刻开销
 - 存储一个指针的每次运算上增加了额外开销
- 基于区域的分配
 - 将一些生命期相同的对象分配在同一个区域中
 - 整个区域同时删除

垃圾回收

- 垃圾
 - 狭义：不能被引用（不可达）的数据
 - 广义：不需要再被引用的数据
 - 基于类型信息，可知道该对象有多大，及该对象的哪些分量包含指向其他对象的引用
 - 假定对对象的引用总是指向该对象的起始位置
 - 把一个用户程序称为增变者，它会修改堆中的对象集合
 - 增变者从存储管理器处获取空间，创建对象，还可引入和消除对已有对象的引用。当增变者不能“到达”某些对象时，这些对象就变成了“垃圾”
- 垃圾回收：自动回收不可达数据的机制，解除了程序员的负担
- 使用的语言
 - Java、Perl、ML、Modula-3、Prolog、Smalltalk

可达性

- 直观地讲，可达性就是指一个存储块可以被程序访问到
- 根集：不需要指针解引用就可以直接访问的数据
 - Java：静态成员、栈中变量
- 可达性
 - 根集的成员都是可达的
 - 对于任意一个对象，如果指向它的一个指针被保存在可达对象的某字段中、或数组元素中，那么这个对象也是可达的
- 性质
 - 一旦一个对象变得不可达，它就不会再变成可达的

垃圾回收方法分类

- 跟踪相关操作，捕获对象变得不可达的时刻，回收对象占用的空间
- 在需要时，标记出所有可达对象、回收其它对象

垃圾回收器的设计目标

- 基本要求
 - 语言必须是类型安全的：保证回收器能够知道数据元素是否为一个指向某内存块的指针
 - 一种语言中，如任何数据分量的类型都是可确定的，则这种语言就称为类型安全的
 - 如果一个语言既不是静态类型安全的，又不是动态类型安全的，就称为不安全的。类型不安全的语言不适合使用自动垃圾回收机制
 - 类型不安全的语言：C，C++
- 性能目标
 - 总体运行时间：不显著增加应用程序的总运行时间
 - 空间使用：最大限度地利用可用内存
 - 停顿时间：当垃圾回收机制启动时，可能引起应用程序的停顿。这个停顿应该比较短
 - 程序局部性：改善空间局部性和时间局部性

改变可达对象集合的操作

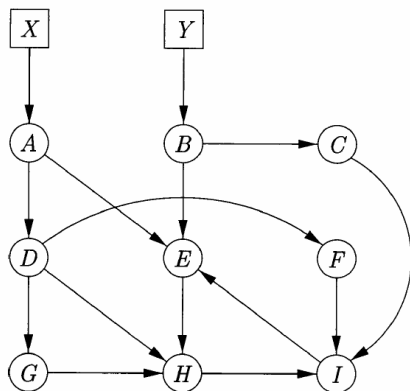
- 对象分配：返回一个指向新存储块的引用
 - 操作由存储管理器完成，向可达对象集中添加成员
- 参数传递/返回值：对象引用从实在参数传递到形式参数，从返回值传递给调用者
 - 这些引用指向的对象仍然是可达
- 引用赋值： $u=v$ ； v 的引用被复制到 u 中， u 中原来的引用丢失。可能使得 u 原来指向的对象变得不可达，并且递归地使得更多对象变得不可达
 - u 现在是 v 所指对象的一个引用，只要 u 可达， v 也可达。 u 中原来的引用丢失了
- 过程返回：活动记录出栈，局部变量消失，根集变小；可能使得一些对象变得不可达

基于引用计数的垃圾回收器

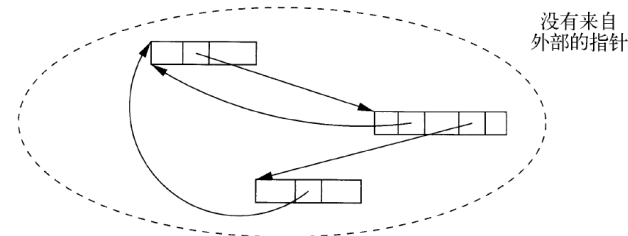
- 每个对象有一个用于存放引用计数的字段，并按照如下方式维护
 - 对象分配：引用计数设为1
 - 参数传递：引用计数加1
 - 引用赋值： $u=v$ ： u 指向的对象引用减1、 v 指向的对象引用加1
 - 过程返回：局部变量指向对象的引用计数减1
- 如果一个对象的引用计数为0，在删除对象之前，此对象中各个指针所指对象的引用计数减1
- 回收器有缺陷，可能引起内存泄漏
 - 不能回收不可达的循环数据结构
- 开销较大、但是不会引起停顿

引用计数的例子

- 考虑如下操作:
 - $y = x$
 - y 是当前函数 f 的局部变量, 且 f 返回
- 修改计数后总是先考虑是否释放
- 释放一个对象之前总是先处理对象内部的指针



引用计数的例子



基于跟踪的垃圾回收

- 功能:
 - 不在垃圾产生时就回收, 而是周期性运行
 - 寻找不可达对象并收回它们的空间
 - 通常是空闲空间被耗尽或空闲空间数量低于某个阈值时启动
- 标记-清扫式垃圾回收
- 标记-清扫式垃圾回收的优化
- 标记并压缩垃圾回收

标记-清扫式垃圾回收

- 一种直接的、全面停顿的算法
- 分成两个阶段
 - 标记: 从根集开始, 跟踪并标记出所有可达对象
 - 清扫: 遍历整个堆区, 释放不可达对象
 - ▣ 找出所有不可达的对象, 将它们放入空闲空间列表
- 如果我们把数据对象看作顶点, 引用看作有向边, 那么标记的过程实际上是从根集开始的图遍历的过程

标记-清扫式垃圾回收

- 该算法主要包括两步
 - mark, 从 root 开始进行树遍历, 每个访问的对象标注为「使用中」
 - sweep, 扫描整个内存区域, 对于标注为「使用中」的对象去掉该标志, 对于没有该标注的对象直接回收掉

标记-清扫式垃圾回收

- 输入: 一个由对象组成的根集, 一个堆和一个被称为Free的包含了堆中所有未分配存储块的空闲空间列表. 所有空间块都用边界标记进行标识, 指明它们的空闲 / 已用状态和大小.
- 输出: 在删除了所有垃圾之后的经过修改的Free列表

标记-清扫垃圾回收算法

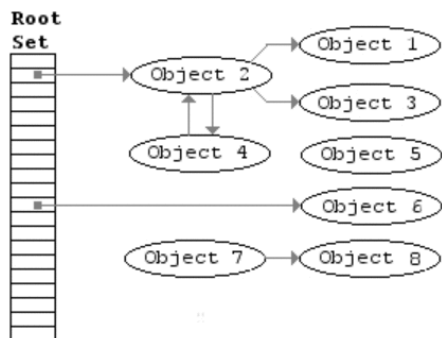
```

/* 标记阶段 */
1) 把被根集引用的每个对象的 reached 位设置为 1, 并把它加入
   到 Unscanned 列表中;
2) while (Unscanned ≠ ∅) {
3)   从 Unscanned 列表中删除某个对象 o;
4)   for (在 o 中引用的每个对象 o' ) {
5)     if (o' 尚未被访问到; 即它的 reached 位为 0) {
6)       将 o' 的 reached 位设置为 1;
7)       将 o' 放到 Unscanned 中;
8)     }
9)   }
10) }
/* 清扫阶段 */
11) Free = ∅;
12) for (堆区中的每个内存块 o) {
13)   if (o 未被访问到, 即它的 reached 位为 0) 将 o 加入到 Free 中;
14)   else 将 o 的 reached 位设置为 0;
15) }

```

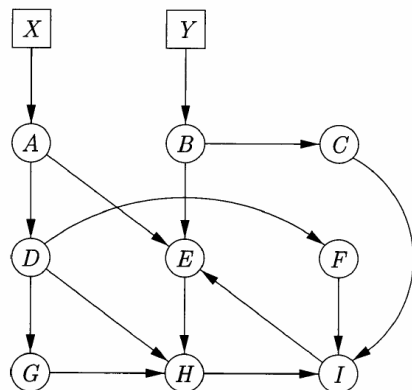
因为语言是强类型的, 所以垃圾回收机制可以知道每个数据对象的类型, 以及这个对象有哪些字段是指针

标记-清扫垃圾回收算法



例子

- 假设 x 是全局变量, y 是当前函数活动的局部变量
- 当前活动返回之后, 进行标记清扫
 - A, D, E, F, I, G, H
 - B, C 不可达



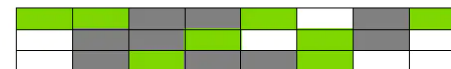
标记-清扫式垃圾回收

该算法的缺点有

- 在进行 GC 期间, 整个系统会被挂起 (暂停, Stop-the-world), 所以在一些实现中, 会采用各种措施来减少这个暂停时间
- heap 容易出现碎片。实现中一般会进行 move 或 compact。(需要说明一点, 所有 heap 回收机制都会这个问题)
- 在 GC 工作一段时间后, heap 中连续地址上存在 age 不同的对象, 这非常不利于引用的本地化 (locality of reference)
- 回收时间与 heap 大小成正比

标记-清扫垃圾回收算法

回收前状态:



回收后状态:



存活对象

可回收

未使用

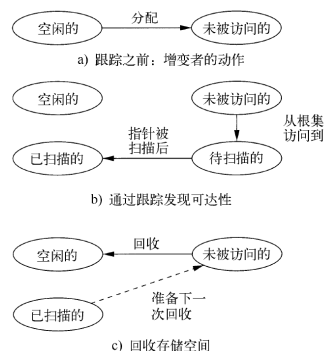
讨论: 垃圾收集的代价

效率!

- 标记-清扫垃圾回收算法采用一种类似于深度优先遍历的算法, 完成整个标记的过程。
- 深度优先搜索所需的时间与它标记的节点个数成正比, 即与可到达数据的数量成正比, 可以推知, 清扫阶段所需的时间, 与堆的大小成正比。假设大小为 H 的堆中有 R 个字的可达数据。
- 垃圾回收一次的代价则为: $aR + bH$, 其中 a 与 b 是常数。
- 回收一次得到的可用字空间大小为: $H - R$ 。
- 那么, 回收到一个可用字空间的代价为 $(aR + bH) / (H - R)$

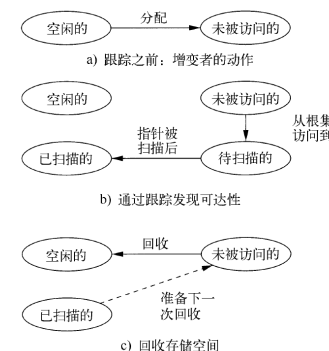
基本抽象分类

- 所有基于跟踪的算法都计算可达对象集合，然后取这个集合的补集
- 使用方式如下：
 - 程序（增变者）运行并发出分配请求
 - 垃圾回收器通过跟踪揭示可达性
 - 垃圾回收器收回不可达对象的存储空间



基本抽象分类

- 每个存储块处于四种状态之一
 - 空闲：可被分配，不存放可达对象
 - 未被访问：默认不可达，未确定可达性前，都标记为未被访问
 - 待扫描：存储块可达，但块中指针还没被扫描，则处于待扫描状态
 - 已扫描：只包含指向其他已扫描或待扫描对象的引用
- 对存储块的操作会改变存储块的状态
 - 应用程序分配
 - 垃圾回收器访问、扫描
 - 收回



标记-清扫垃圾回收算法的优化

- 基本算法需要扫描整个堆，代价很大
- 优化
 - 用一个列表记录所有已经分配的对象
 - 不可达对象等于已分配对象减去可达对象
 - 已分配对象和可达对象之间的差集
- 好处
 - 只需要扫描这个列表就可以完成清扫
- 坏处
 - 需要维护这个列表

标记-清扫垃圾回收算法的优化

- Baker的标记—清扫式回收器
 - 输入：一个由对象组成的根集，一个堆区，一个空闲列表Free，一个名为Unreached的已分配对象的列表
 - 输出：经过修改的Free列表和Unreached列表。Unreached列表保存了被分配的对象

标记-清扫垃圾回收算法的优化

- 方法
 - Free, Unreached, Unscanned, Scanned的四个列表，分别保存了处于空闲，未被访问，待扫描和已扫描状态上的所有对象
 - 假定每个对象中都包含了一些二进制位，指明该对象处于上述四个状态的哪一个
 - 最初，Free就由存储管理器维护的空闲列表
 - 所有已分配的对象都在Unreached列表中〔这个表同时也由存储管理器在为对象分配存储块时维护〕

优化后的算法

- 使用了四个列表：Scanned, Unscanned, Unreached, Free

```

1)  Scanned = ∅;
2)  Unscanned = 在根集中引用的对象的集合;
3)  while (Unscanned ≠ ∅) {
4)      将对象从 Unscanned 移动到 Scanned;
5)      for (在 o 中引用的每个对象 o') {
6)          if (o' 在 Unreached 中)
7)              将 o' 从 Unreached 移动到 Unscanned 中;
8)      }
9)  Free = Free ∪ Unreached;
    Unreached = Scanned;

```

压缩并标记垃圾回收

- 存在多种进行重新定位的回收器，不同之处在于它们是在本地进行重新定位还是在重新定位前预留了空间
 - 本节介绍的压缩并标记垃圾回收器在本地压缩对象，属于本地重新定位
 - 拷贝回收器，把对象从内存的一个区域移到另一个区域
 - 保留额外的空间用于重新定位可使得一发现可达对象就立刻移动它
- 对可达对象进行重定位可以消除存储碎片
 - 把可达对象移动到堆区的一端，另一端则是空闲空间
 - 空闲空间合并成单一块，分配内存时高效率
 - 可达对象占用的空间要大大小于空闲空间，标记出所有“窗口”后可以将所有可达对象重新定位到堆区的一端，使堆区的所有空闲空间成为一个块

压缩并标记垃圾回收

```

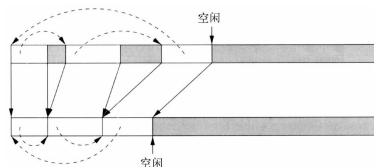
/* 标记 */
1)  Unscanned = 根集引用的对象的集合;
2)  while (Unscanned ≠ ∅) {
3)    从 Unscanned 中移除对象 o;
4)    for (在 o 中引用的每个对象 o') {
5)      if (o' 是未访问的) {
6)        将 o' 标记为已访问的;
7)        将 o' 加入到列表 Unscanned 中;
      }
    }
8)  /* 计算新的位置 */
   free = 堆区的开始位置;
9)  for (从低端开始，遍历堆区中的每个存储块 o) {
10)    if (o 是已访问的) {
11)      NewLocation(o) = free;
12)      free = free + sizeof(o);
    }
  }

```

```

/* 重新设置引用目标并移动已被访问的对象 */
13) for (从低端开始，堆区中的每个存储块 o) {
14)   if (o 是已访问的) {
15)     for (o 中的每个引用 o.r)
16)       o.r = NewLocation(o.r);
17)     将 o 拷贝到 NewLocation(o);
   }
18) for (根集中的每个引用 r)
19)   r = NewLocation(r);

```



拷贝回收器

- 预先保留了可将对象移入的空间，解除了跟踪和发现空闲空间间的依赖关系
- 堆空间被分为两个半空间
 - 增变者在某个半空间内分配存储，当充满这个半空间时，开始垃圾回收
 - 回收时，可达对象被拷贝到另一个半空间
 - 回收完成后，两个半空间角色对调
 - 增变者可继续运行，并在半空间B中分配对象
 - 下一轮垃圾回收将把可达对象移动到A

压缩并标记垃圾回收

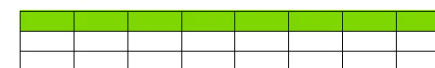
- 整个过程分成三个步骤
 - 标记阶段，参考之前的标记过程
 - 计算新位置
 - 算法扫描堆区中的已分配内存段，并为每个可达对象计算新的地址
 - 新地址从堆的最低端开始分配，可达对象之间没有空闲存储窗口
 - 每个对象的新地址记录在一个名为NewLocation的结构中
 - 移动并设置新的引用
 - 算法将对象拷贝到它们的新地址，更新对象中的所有引用
 - 使之指向新地址，新地址可在NewLocation中找到

压缩并标记垃圾回收

回收前状态：



回收后状态：



存活对象

可回收

未使用

拷贝回收器

- 拷贝回收算法虽然能解决内存碎片问题，但是需要多次遍历heap空间，这会导致较大性能损耗
- 拷贝回收算法采用空间换时间的方式来提升性能
 - 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
 - 输出：最后,To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。From半空间此时全部空闲
 - 方法：在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间
- 这类 GC 并不会真正去“回收”不可到达对象，而是会把所有可到达对象移动到一个区域，heap 中剩余的空间就是可用的了（因为这里面都是垃圾）

拷贝回收器

- 典型的代表半空间拷贝回收器（semispace collector）。其工作过程是这样的
 - heap 被分成2份相邻的空间（semispace）：fromspace 与 tospace
 - 在程序运行时，只有 fromspace 会被使用（分配新对象）
 - 在 fromspace 没有足够空间容纳新对象时，程序会被挂起，然后把 fromspace 的可到达对象拷贝到 tospace
 - 在拷贝完成时，之前的2个空间交换身份，tospace 成了新一轮的 fromspace

拷贝回收器

```
1) CopyingCollector () {
2)     for (From 空间中的所有对象 o) NewLocation(o) = NULL;
3)     unscanned = free = To 空间的开始地址;
4)     for (根集中的每个引用 r)
5)         将 r 替换为 LookupNewLocations(r);
6)     while (unscanned ≠ free) {
7)         o = 在 unscanned 所指位置上的对象;
8)         for (o 中的每个引用 o.r)
9)             o.r = LookupNewLocation(o.r);
10)        unscanned = unscanned + sizeof(o);
11)    }
12)    /* 如果一个对象已经被移动过了，查找这个对象的新位置 */
13)    /* 否则将对象设置为待扫描状态 */
14)    LookupNewLocation(o) {
15)        if (NewLocation(o) = NULL) {
16)            NewLocation(o) = free;
17)            free = free + sizeof(o);
18)            将对象 o 拷贝到 NewLocation(o);
19)        }
20)        return NewLocation(o);
21)    }
```

分代回收

- 针对很多变量存在时间较短的特点设计
- 基本思想：将堆区进行切割成块并标号，垃圾回收技术每次在较低标号的块上进行
- 具体执行方案：
 - 新对象在编号较低的堆块上创建
 - 每次在0号堆块被填满时刻，就对0号堆块进行垃圾回收；回收完毕后，将i号堆块往后移动到i+1号堆块中
 - 维护一个记忆集，用以存储较高号的堆块对较低号堆块中对象的引用

拷贝回收器

- 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
- 输出：最后,To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。Fron半空间此时全部空闲
- 方法：
 - 算法在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间

拷贝回收器

回收前状态：

回收后状态：

存活对象

可回收

未使用

保留区域

分代回收

- 采用该策略的基本原因：
 - 因为较年轻的世代往往包含较多的垃圾，也就更频繁地被回收
 - 编程实践中，极少出现较老世代对较新世代的引用。因此，我们只需要记录该种情况，并定期采用其他策略进行处理即可

GC的扩充内容

- 为什么需要 GC?
 - 计算机诞生初期，在程序运行过程中没有栈帧（stack frame）需要去维护
 - 早期内存采取的是静态分配策略，这虽然比动态分配要快，但是其一明显的缺点是程序所需的数据结构大小必须在编译期确定，而且不具备运行时分配的能力

GC的扩充内容

- GC的历史
 - C/C++/Pascal 把堆管理这个任务交给了程序员，但事实证明这非常容易出错，野指针（wild pointer）、悬挂指针（dangling pointer）是比较典型的错误
 - 实际应用的场景中，动态分配的对象传入了其他过程中，这时程序员或编译器就无法预测这个对象什么时刻不再需要，现如今的面向对象语言，这种场景更是频繁，这也就间接促进了自动内存管理技术的发展
 - 实际上，即便是 C/C++，也有类似 Boehm GC 这样的第三方库来实现内存的自动管理

JAVA的GC设计

引用类型	被垃圾回收时间	用途	生存时间
强引用	从来不会	对象的一般状态	JVM停止运行时终止
软引用	当内存不足时	对象缓存	内存不足时终止
弱引用	正常垃圾回收时	对象缓存	垃圾回收后终止
虚引用	正常垃圾回收时	跟踪对象的垃圾回收	垃圾回收后终止

GC的扩充内容

- GC的历史
 - Algol-58 语言首次提出了块结构（block-structured），通过在内存中申请栈帧来实现按需分配的动态策略
 - 在过程被调用时，帧（frame）会被压到栈的最上面，调用结束时弹出。栈分配策略赋予程序员极大的自由度，局部变量在不同的调用过程中具有不同的值，这为递归提供了基础
 - 后进先出（Last-In-First-Out, LIFO）的栈限制了栈帧的生命周期不能超过其调用者，而且由于每个栈帧是固定大小，所以一个过程的返回值类型也必须在编译期确定。所以诞生了新的内存管理策略——堆管理

GC的实现常见策略

- 追踪（Tracing）
 - 标记清扫（Mark-Sweep）
 - 标记压缩（Mark-Compact）
 - 拷贝（Copying）
- 引用计数（Reference counting）

JAVA的GC设计

- 强引用(StrongReference)
 - 强引用是使用最普遍的引用。如果一个对象具有强引用，那垃圾回收器绝不会回收它

```
Object strongReference = new Object();
public void test() {
    Object strongReference = new Object();
    // 省略其他操作
}
```

- 显式地设置strongReference对象为null，或让其超出对象的生命周期范围，则gc认为该对象不存在引用，这时就可以回收这个对象。具体什么时候收集这要取决于GC算法。

JAVA的GC设计

• 软引用(SoftReference)

- 如果一个对象只具有软引用，则内存空间充足时，垃圾回收器就不会回收它；如果内存空间不足了，就会回收这些对象的内存。只要垃圾回收器没有回收它，该对象就可以被程序使用
- 软引用可用来实现内存敏感的高速缓存

```
// 强引用
String strongReference = new String("abc");
// 软引用
String str = new String("abc");
SoftReference<String> softReference = new SoftReference<String>(str);
```

JAVA的GC设计

• 软引用应用场景：浏览器的后退按钮

- 如果一个网页在浏览结束时就进行内容的回收，则按后退查看前面浏览过的页面时，需要重新构建
- 如果将浏览过的网页存储到内存中会造成内存的大量浪费，甚至会造成内存溢出

JAVA的GC设计

• 弱引用(WeakReference)

- 弱引用与软引用的区别在于：只具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存
- 由于垃圾回收器是一个优先级很低的线程，因此不一定会很快发现那些只具有弱引用的对象

```
String str = new String("abc");
WeakReference<String> weakReference = new WeakReference<>(str);
String strongReference = weakReference.get();
```

JAVA的GC设计

• 软引用(SoftReference)

- 注意：软引用对象是在jvm内存不够的时候才会被回收，我们调用System.gc()方法只是起通知作用，JVM什么时候扫描回收对象是JVM自己的状态决定的。就算扫描到软引用对象也不一定会回收它，只有内存不够的时候才会回收
- 垃圾收集线程会在虚拟机抛出OutOfMemoryError之前回收软引用对象，而且虚拟机会尽可能优先回收长时间闲置不用的软引用对象。对那些刚构建的或刚使用过的**较新的软对象会被虚拟机尽可能保留**

JAVA的GC设计

• 软引用应用场景：浏览器的后退按钮

```
// 获取浏览器对象进行浏览
Browser browser = new Browser();
// 从后台程序加载浏览页面
BrowserPage page = browser.getPage();
// 将浏览完毕的页面置为软引用
SoftReference softReference = new SoftReference(page);

// 后退或者再次浏览此页面时
if(softReference.get() != null) {
    // 内存充足，还没有被回收器回收，直接获取缓存
    page = softReference.get();
} else {
    // 内存不足，软引用的对象已经回收
    page = browser.getPage();
    // 重新构建软引用
    softReference = new SoftReference(page);
}
```

JAVA的GC设计

• 虚引用(PhantomReference)

- 与其他几种引用都不同，虚引用并不会决定对象的生命周期。如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收器回收
- 虚引用与软引用和弱引用的一个区别在于：虚引用必须和引用队列(ReferenceQueue)联合使用。当垃圾回收器准备回收一个对象时，如果发现它还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中

- 虚引用应用场景：虚引用主要用来跟踪对象被垃圾回收器回收的活动
 - 与其他几种引用都不同，虚引用并不会决定对象的生命周期。如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收器回收

```
String str = new String("abc");
ReferenceQueue queue = new ReferenceQueue();
// 创建虚引用，要求必须与一个引用队列关联
PhantomReference pr = new PhantomReference(str, queue);
```

第五章 运行时环境

冯洋

南京大学计算机学院
fengyang@nju.edu.cn